BROWSER DEVTOOLS INTENSIVE

브라우저 개발자 도구 실전 활용

Console · Elements · 자동화 · LLM 연동 · 확장 프로그램

PROGRAM
Browser DevTools

SESSION
전체 과정 (2hr)

GOAL
DevTools 실전 자동화

FORMAT
강의 + 실습

대상
비개발자 포함 전 직군

준비물
Chrome 브라우저

SESSION 01 · AGENDA

강의 구성

1장	**Console 기초 + 패널 소개** DevTools 열기 / Elements / Console / 기초 실습	20분
2장	**Console 대표 유형 4가지** 폼 채우기 · 다운로드 · 시각화 · LLM 연동	70분
3장	**개인 확장 프로그램 등록** manifest · content.js · 직접 등록 실습	20분
Q&A	**마무리 & 다음 강의 예고** Network 패널 예고 / 질의응답	10분

1장 — CONSOLE 기초 + 패널 소개

개발자 도구 열기

HOW TO OPEN 단축키

F12 / 우클릭 → 검사

Mac: Cmd+Option+I

오늘은 Console + Elements

두 탭만 사용합니다

ELEMENTS PANEL 설계도 탐색

HTML 태그 트리 구조 확인

요소 클릭 → 화면 하이라이트

텍스트·색상 실시간 수정

(새로고침하면 원복)

CONSOLE PANEL 미니 개발환경

JS 코드 즉시 실행

에러·경고 메시지 확인

console.log()로 값 출력

현재 페이지 모든 요소 접근

KEY TAKEAWAY

Elements는 페이지의 [설계도]를 보는 창, Console은 그 위에서 코드를 실행하는 [엔진]입니다. 두 패널을 함께 쓰면 강력해집니다.

1장 — ELEMENTS 패널 실습

팝업 정보 읽기

항목	값	의미
요소명	i.MyView-module__naver_logo___Y42	선택된 HTML 요소 클래스명 (i 태그 = 이미지 계열)
크기	72 × 14 px	실제 렌더링 크기
Margin	3px 8px 0px 0px	상·우·하·좌 (CSS Box Model 기준)
Name	(비어 있음)	접근성 레이블 — 스크린리더가 읽는 이름
Role	generic	의미 없는 구조 요소를 뜻함
Keyboard-focusable	⊘ 불가	키보드 Tab 포커스 이동 불가



INSIGHT · 접근성 관점

Role이 generic이고
키보드 포커스 불가
→ 스크린리더 사용자에게
이 로고가 인식되지 않음

실무 개선 포인트:
role="img" 추가
aria-label="NAVER" 설정
focusable 여부 검토

→ 심화 주제로 연결 가능

1장 — CONSOLE 기초 코딩

따라 치기 3단계

STEP 01 첫 출력

```
console.log("안녕하세요!")
```

POINT

콘솔에 값이 출력됨을 확인

STEP 02 페이지 정보

```
console.log(document.title)
console.log(location.href)
```

POINT

document·location 객체 개념 도입

STEP 03 팝업 연동

```
const name = prompt("이름을 입력하세요");
alert("환영합니다, " + name + "님!");
```

POINT

변수·문자열 연결·팝업 함수 체험

PITFALL

alert을 반복문 안에 많이 넣으면 브라우저가 멈춘 것처럼 보일 수 있음. 실습은 about:blank 페이지에서 권장.

SESSION 02 · ONE SENTENCE

설치도 없이,
브라우저 하나로
이런 것까지 됩니다.

2

01 · 폼 자동 채우기

02 · 다운로드 자동화

03 · 데이터 시각화

04 · LLM 연결 분석

유형 01 — FORM AUTO-FILL

폼 자동 채우기

USE CASE

반복적인 회원가입·설문·신청서 작성
RPA 도구 없이 브라우저에서 바로 처리

```
// 입력 필드 찾아 값 넣기
document.querySelector('#username')
  .value = '홍길동';

document.querySelector('#email')
  .value = 'hong@example.com';

// 버튼 자동 클릭
document.querySelector('#submit')
  .click();
```

PRINCIPLE

querySelector()로 HTML 요소를 선택
.value = '...' 로 값 직접 주입
.click()으로 버튼 자동 클릭

PRACTICE

about:blank에 폼 HTML을 직접 만들어 실습
또는 테스트용 사이트 활용
querySelectorAll로 여러 필드 한번에 처리

REAL WORLD

공공기관 신청서·반복 테스트에서 시간 절약

TIME SAVING

수동으로 10분 걸리던
반복 입력 작업을

10초

로 줄일 수 있습니다

10초

PITFALL

동적으로 로딩되는 폼은 setTimeout() 또는
MutationObserver 조합 필요

유형 02 — BULK DOWNLOAD

한 번에 다운로드하기

USE CASE

이미지·PDF 링크가 수십 개 — 일일이 저장하는 대신 코드 한 번으로 처리

```
// 모든 이미지 URL 출력
document.querySelectorAll('img')
  .forEach(img => console.log(img.src));
```

```
// PDF 링크만 추출
document.querySelectorAll('a[href$=".pdf"]')
  .forEach(a => console.log(a.href));
```

STEP 1 — URL 추출

querySelectorAll()로 원하는
요소 목록 선택 후 URL 출력

STEP 2 — 목록 복사

출력된 URL을
다운로드 매니저에 붙여넣기

ADVANCED · 자동 다운로드 트리거

```
document.querySelectorAll('a[href$=".pdf"]')
 .forEach(a => {
  const link = document.createElement("a");
  link.href = a.href;
  link.download = "";
  link.click();
 });
```

⚠ PITFALL

저작권·이용약관 주의. 개인 학습·업무용 데이터에 한정 사용 권장.

유형 03 — DATA VISUALIZATION

팝업창에 데이터 시각화하기

USE CASE

페이지 데이터를 즉석에서 표·차트로 — 설치 없이 임시 대시보드 생성

```
const popup = window.open('','','width=600,height=400');
popup.document.write(`
  <h2>페이지 분석</h2>
  <table border="1">
    <tr><td>링크 수</td>
        <td>${document.querySelectorAll('a').length}</td></tr>
    <tr><td>이미지 수</td>
        <td>${document.querySelectorAll('img').length}</td></tr>
  </table>
```

ADVANCED · Chart.js CDN 연동

```
popup.document.write(`
 <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
 <canvas id="chart"></canvas>
 <script>new Chart(...)</script>
```

PRINCIPLE

window.open()으로 새 창 생성 → document.write()로 HTML 직접 삽입
페이지 DOM 데이터를 템플릿 리터럴(` `)로 실시간 주입

WHAT YOU CAN BUILD

링크·이미지 수 요약 표
텍스트 키워드 빈도 차트
가격·수치 비교 그래프

KEY LIMIT

새로고침하면 사라짐
결과 저장 시
별도 복사 필요

실습 아이디어

뉴스 사이트 접속 → 기사 제목 전체를 팝업 표로 추출
쇼핑몰 접속 → 상품명·가격 리스트 팝업 시각화
Chart.js 연동 → 요소 수를 막대 그래프로 표현

유형 04 — LLM INTEGRATION

LLM 연결하여 분석하기

방식 A · 외부 API (OpenAI / Claude)

```
const text = document.body.innerText.slice(0,1000);

const res = await fetch(
  "https://api.openai.com/v1/chat/completions",
  { method:"POST",
    headers:{ "Authorization":"Bearer KEY" },
    body: JSON.stringify({
      model:"gpt-4o",
      messages:[{role:"user",
        content:"3줄 요약:\n"+text}]
    })
```

방식 B · Local LLM (Ollama)

```
// ollama run llama3 으로 로컬 서버 실행
const text = document.body.innerText.slice(0,1000);

const res = await fetch(
  "http://localhost:11434/api/generate",
  { method:"POST",
    body: JSON.stringify({
      model:"llama3",
      prompt:"3줄 요약:\n"+text,
      stream:false
    })
```

항목	외부 API	Local LLM
설치	없음	Ollama 설치 필요
비용	사용량 과금	무료
성능	높음 (GPT-4o 등)	모델에 따라 다름
보안	데이터 외부 전송	완전 로컬 처리
인터넷	필요	불필요

⚠ API Key를 Console에 직접 입력하면 화면에 노출됩니다 — 실습용 Key 사용 후 즉시 폐기 권고

3장 — CHROME EXTENSION

개인 확장 프로그램 등록하기

manifest.json 확장 프로그램 설명서

```
{
  "manifest_version": 3,
  "name": "내 첫 확장",
  "version": "1.0",
  "permissions":
    ["activeTab","scripting"],
  "action": {
    "default_popup":"popup.html"
  }
}
```

content.js 페이지에 실행할 코드

```
// 앞에서 만든 기능을
// 모두 여기에 담습니다

document.querySelector
  ('#username')
  .value = '홍길동';

// LLM 호출, 다운로드
// 모두 여기서 실행
```

popup.html 아이콘 클릭 시 UI

```
<button id="fill">
  폼 자동 채우기
</button>
<button id="dl">
  다운로드
</button>
<button id="ai">
  AI 요약
</button>
```

등록 방법 4단계

01 파일 3개를 하나의 폴더에 저장

02 chrome://extensions 접속

03 개발자 모드 토글 ON

04 압축해제된 확장 프로그램 로드 → 폴더 선택

완료 → 브라우저 우상단에 아이콘이 생기고, 클릭 한 번으로 앞에서 만든 모든 기능을 실행할 수 있습니다.

오늘 배운 것

DevTools 하나로 이만큼 됩니다.

1장	Console + Elements 패널 구조 이해 + 기초 실습
2장	폼 채우기 · 다운로드 · 시각화 · LLM 연동 4가지 유형
3장	확장 프로그램으로 패키징 → 버튼 하나로 실행
Next	Network 패널 → API 트래픽 분석 + 심화 자동화